Integrating Vector Databases across Embedding Models

BEINING YANG, University of Edinburgh, UK
YANG CAO, University of Edinburgh, UK
YANG REN, Edinburgh Research Center, Central Software Institute, Huawei, UK

Vector databases have been widely used to implement similarity search over unstructured objects, *e.g.,* documents and images. Each vector database is produced by an embedding model that encodes the objects in a way such that more similar objects are embedded to closer vectors, allowing us to use top-k vector search as an implementation of top-k object similarity search. It is common practice that different vector databases use distinct embedding models and the same object may be encoded by different embedding vectors across databases. As a result, one cannot share and integrate vector databases to expand similarity search across datasets, a property we take for granted for relational databases. In this work, we attempt to break the barrier between different vector databases, by developing an approach to integrating vector databases generated by different embedding models, with neither any access to the encoded data objects nor knowledge of the embedding models. Our approach is rooted in the *local isometry hypothesis*, a finding made via extensive experiments on real-life embedding vectors, and is backed up by theoretical analysis that bounds the quality of integrated vector database. Experimental results show that we can integrate vector databases produced by various popular embedding models, *e.g.,* NV-embed-V2, OpenAI Ada, GloVe, Mistral and FastText, while offering high recall of top-k similarity search over the integrated datasets.

CCS Concepts: • **Information systems → Retrieval models and ranking**; **Information integration**; • **Computing methodologies → Machine learning**.

Additional Key Words and Phrases: Vector databases; Data integration; Cross-model vector database integration

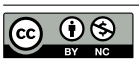
ACM Reference Format:
Beining Yang, Yang Cao, and Yang Ren. 2025. Integrating Vector Databases across Embedding Models. *Proc. ACM Manag. Data* 3, 6 (SIGMOD), Article 338 (December 2025), 28 pages. https://doi.org/10.1145/3769803

1 Introduction

A vector database is a vector representation of a collection of unstructured data items, *e.g.,* text and images, such that more similar items are encoded by vectors that are closer in the vector space. This allows us to use vector search over the encoded vectors to implement similarity search over unstructured data. The mapping from data items to vectors is done via an embedding model, which is often part of a neural network that is capable of accurately capturing the extent of dissimilarity between data items via distances between the corresponding vectors that encoding them.

Advancements in embedding models have made vector search highly effective for text [14, 50, 65, 77, 92] and image [33, 34, 42, 81] retrival. This has led to widespread adoption of vector databases in applications like recommender systems [52, 75, 89, 90] and retrieval-augmented generation (RAG) for LLMs [22, 25, 73], where searches run constantly over curated embedding vectors.

Authors' Contact Information: Beining Yang, B.Yang-32@sms.ed.ac.uk, University of Edinburgh, Edinburgh, UK; Yang Cao, yang.cao@ed.ac.uk, University of Edinburgh, Edinburgh, UK; Yang Ren, Edinburgh Research Center, Central Software Institute, Huawei, Edinburgh, UK, renyang1@huawei.com.



ACM 2836-6573/2025/12-ART338
https://doi.org/10.1145/3769803

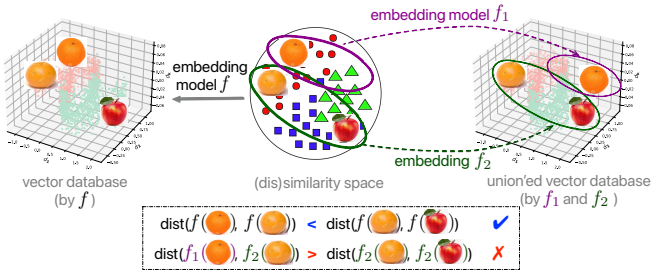


Fig. 1. What happens when we union two vector databases? (The union of vector databases embedded by f_1 and f_2 is not a vector database of the dataset that would be embedded by f.)

Vector databases cannot share data. Although vector databases are a widely accepted terminology, they lack one of the fundamental properties of traditional databases – data sharing. Sharing data in the form of a database across different parties has been an instrumental property taken for granted by relational database system users. The property was even explicitly highlighted by System R, the first database system [6] ever built. This allows us to collect and integrate data from multiple sources databases [23, 28, 35, 64], and query the integrated data for a complete answer that covers all sources.

However, it is not so straightforward to share vector databases as they each is "encrypted" by the embedding model that generates the vectors. The same data item is thus encoded differently across vector databases generated by different embedding models. As a result, it does not make any sense to union two vector databases, although this would enable us to share and integrate vector databases from different embedding models of varying data sources. As illustrated in Fig. 1, simply taking the union of two vector databases embedded by different models would render vector search useless for similarity search over the integrated dataset, as it breaks the fundamental principle that more similar items should be encoded by closer vectors.

Indeed, it is commonplace that different vector databases are embedded by distinct, customized and fine-tuned embedding models. As pointed out by major industry vendors, finetuning embedding models on in-domain data can significantly improve vector search and RAG accuracy, and significant industrial effort has been devoted to make customizing embedding models efficient and accessible to public users [20]. In addition, a substantial part of unstructured data are generated on edge devices which use customized device-specific embedding models [15]. These diverse embedding models produce a vast number of isolated, non-exchangeable vector databases.

Our vision. We aim to realize the vision of sharing and integrating vector databases from *different* embedding models, *even if we have access only to the vectors and are oblivious to the embedding models*. While data integration has long been a classic topic in our community and has been studied for decades, it primarily deals with relational databases and does not help with vector databases, which lack database schemas and use only numeric values that carry no explicit semantics.

Impact. We envisage that realizing this vision could benefit a range of directions, from practical industrial settings to data sharing scenarios where explicit record sharing is either prohibited or inefficient, yet there is still a need for collective searches and analyses. Below are a few examples.

(A1) *Embedding backfilling*. Aligning and integrating vectors across embedding models can help with the embedding backfilling problem. As noted in [38], in industrial settings, the embedding model is constantly updated, resulting in embedding vectors that are incompatible with historical records over time. Moreover, historical versions of the embeddings can never be retired as they remain used by legacy applications. As such, we need a system-wide overhaul to backfill vectors and unify with all applications. Our vision could potentially help with this emerging industrial

challenge [1], by making embedding vectors "talk" with each other without backfills.

(A2) *Privacy-preserving federated image/document search.* Privacy-preserving searching over unstructured datasets such as images and documents managed by autonomous parties is challenging, if not impossible at all. While privacy-preserving federated learning [45] offers various privacy controls, it enforces all parties to use the same learned embedding model. Realizing our vision would allow each autonomous party to use its own embedding model while still offering search over the entire collection of datasets from all parties, without actually moving or revealing their data records.

(A3) *Cloud-edge similarity search.* It is common to have data residing in both edge devices and the cloud, each independently embedded by different embedding models. A collaborative search involves finding relevant *e.g.,* images in the cloud that are similar to a query image on an edge device. Sending the query image to the cloud would incur heavy communication costs and may not be feasible due to privacy concerns. By bridging and integrating on-device embedding vectors with those in the cloud, one would only need to send the query embedding vector for retrieval, which is more secure and cost-efficient than sending the query image directly.

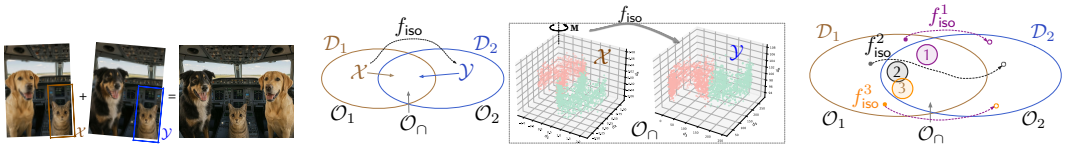
Prior work & challenges. Realizing the vision is challenging. Previous work on vector databases and similarity search mostly focuses on efficiency by designing vector indices [7, 26, 60, 61] and building dedicated vector database systems [11, 32, 67, 83, 86]. They offer little help in dealing with the interoperability between vector databases from different embedding models. As an added challenge, we aim to integrate vector databases by examining the embedding vectors only. This implies that we cannot reconstruct a global vector database by re-embedding data items, nor train a model to learn a mapping between the embedding models from the dataset.

Towards vector database integration. As a first attempt to realize this vision, we present a data-driven approach to aligning and integrating vector databases generated by different embedding models. It is based on a simple hypothesis: if two embedding models capture the same similarity over the same data items, their geometric shapes in the high-dimensional vector space should be approximately isometric. However, via extensive experiments over real-life embedding vectors, we find that the hypothesis does not hold globally but applies to local regions of the vector database where distances between vectors are relatively short. We refer to this phenomenon as the *local isometry hypothesis* and verify its existence in benchmarks.

The hypothesis gives us a method to combine vector databases across models. The idea is to use vectors that encode the same data items across the two vector databases as references. Such references can be obtained via *e.g.,* known correspondence between different embedding versions in backfilling scenarios, data linkage in datasets, or by querying remote embedding models via sampled vectors. We then compute a collection of *local isometries* that align the references across databases, and use them to integrate new vectors that are not references.

We find that the effectiveness of local isometries for transforming and integrating new embedding vectors is strongly connected to their scopes and determining the scopes of local isometries for integrating different vector databases is a central problem underpinning the approach. To tackle this, we prove an upper bound of the integration error of local isometries, which connects the scope of isometries to the singular values, intrinsic dimensions and conditioning of the vector database viewed as a matrix of column vectors. Such connection gives us guidelines for determining the best scopes of isometries of a given pair of vector databases. Using benchmarks, we show that our method can integrate vector databases across major embedding models, *e.g.,* OpenAI Ada, GloVe, Mistral, NV-embed-V2 and FastText, while offering high recall of top-k search over the integrated data.

Organization. In Section 2, we propose the idea of integrating vector databases and present the initial isometry hypothesis. We evaluate the hypothesis in Section 3 and consolidate it into the



(a) Image fusion (b) Hypothesis A: globally isometric (c) Hypothesis B: locally isometric

Fig. 2. Drawing parallel between image correlation in computer vision (a) and vector database integration (b-c): (a) fuses two images into one by taking the cat as reference to compute a rigid motion ($\mathcal{X} \to \mathcal{Y}$) that rotates, reflects, scales and translates the first image and fuses it with the second. (b) Hypothesis A assumes *global consistency* between between $\mathcal{D}_1[\mathcal{O}_\cap]$ and $\mathcal{D}_2[\mathcal{O}_\cap]$; by treating $\mathcal{O}_\cap$ as the cat in (a), we determine an isometry f_{iso} to transform $\mathcal{D}_1$ and integrate it with $\mathcal{D}_2$. (c) Hypothesis B acknowledges that vector databases do not have global geometry consistency for $\mathcal{O}_\cap$; it instead suggests that we exploit multiple local isometries between $\mathcal{D}_1[\mathcal{O}_\cap]$ with $\mathcal{D}_2[\mathcal{O}_\cap]$, enabling integration even if Hypothesis A does not hold.

revised local isometry hypothesis in Section 4.1, based on which we develop our vector database integration framework (Section 4.2). We prove an upper bound on the integration error of the framework (Section 5), which guides its implementation (Section 6). We experimentally verify the effectiveness of our approach with 5 public datasets and 6 major embedding models (Section 7). We discuss additional related research in Section 8 and conclude in Section 9.

2 Integrating Vector Databases

Vector databases. Denote by $\mathcal{U}$ the universe of data items, of which the *dissimilarity* between items is measured by function $\texttt{dsim}$. Let $\mathcal{O}$ be a subset of $\mathcal{U}$. An embedding function $\texttt{emb}$ for $\mathcal{O}$ *w.r.t.* $\texttt{dsim}$ maps items in $\mathcal{O}$ to high-dimensional vectors $\mathbb{R}^d$ such that $\|\mathbf{x}_1 - \mathbf{x}_2\|$ encodes $\texttt{dsim}(o_1, o_2)$ for any $o_1, o_2 \in \mathcal{O}$ with $\texttt{emb}(o_1) = \mathbf{x}_1$ and $\texttt{emb}(o_2) = \mathbf{x}_2$, where $\|\mathbf{x}_1 - \mathbf{x}_2\|$ denotes the distance between $\mathbf{x}_1$ and $\mathbf{x}_2$ in $\mathbb{R}^d$. We refer to $\texttt{emb}(\mathcal{O})$, *i.e.,* $\cup_{o\in\mathcal{O}}\texttt{emb}(o)$, as a *vector database of* $\mathcal{O}$ *with* $\texttt{emb}$ *w.r.t.* $\texttt{dsim}$.

The goal of vector databases is to implement top-k similarity search: given item $o \in \mathcal{O}$, find k most similar items in $\mathcal{O}$ *w.r.t.* $\texttt{dsim}$, by searching for k vectors in $\texttt{emb}(\mathcal{O})$ that are nearest to $\texttt{emb}(o)$.

In practice, vector distances in $\texttt{emb}(\mathcal{O})$ are not exactly equivalent to dissimilarity of embedded items. Hence, $\|\mathbf{x}_1 - \mathbf{x}_2\|$ is only an approximation of $\texttt{dsim}(o_1, o_2)$ as long as top-k vector search over $\texttt{emb}(\mathcal{O})$ implements top-k similarity search sufficiently accurate.

Vector database integration. Let $\mathcal{O}_1$ and $\mathcal{O}_2$ be subsets of $\mathcal{U}$, *i.e.,* two different datasets. Denote by $\mathcal{D}_1$ (resp. $\mathcal{D}_2$) the vector database of $\mathcal{O}_1$ (resp. $\mathcal{O}_2$) with embedding function $\texttt{emb}_1$ (resp. $\texttt{emb}_2$) *w.r.t.* $\texttt{dsim}$. The *integration of* $\mathcal{D}_1$ *and* $\mathcal{D}_2$ is a function $\mathcal{T}$ that takes as input only vectors of $\mathcal{D}_1$ and $\mathcal{D}_2$, and returns an vector database $\mathcal{T}(\mathcal{D}_1, \mathcal{D}_2)$ of $\mathcal{O}_1 \cup \mathcal{O}_2$ with some embedding $\texttt{emb}_\mathcal{T}$ *w.r.t.* $\texttt{dsim}$.

Intuitively, $\mathcal{T}$ seeks to combine separate vector databases $\mathcal{D}_1$ and $\mathcal{D}_2$ into a merged database $\mathcal{D}_\cup$ that encodes $\texttt{dsim}$ of data across $\mathcal{O}_1 \cup \mathcal{O}_2$ via some embedding function $\texttt{emb}_\mathcal{T}$. This implies that, for any items $o, o' \in \mathcal{O}_1 \cup \mathcal{O}_2$, vector distance $\|\texttt{emb}_\mathcal{T}(o) - \texttt{emb}_\mathcal{T}(o')\|$ in $\mathcal{D}_\cup$ is, approximately,

- $\|\texttt{emb}_1(o) - \texttt{emb}_1(o')\|$ if both o and o' are in $\mathcal{O}_1$ (agrees with $\mathcal{D}_1$);
- $\|\texttt{emb}_2(o) - \texttt{emb}_2(o')\|$ if both o and o' are in $\mathcal{O}_2$ (agrees with $\mathcal{D}_2$);
- $\texttt{dsim}(o, o')$ if o and o' are from different datasets.

If this is doable, it would allow us to construct a *global* vector database for a federation of disparate datasets without accessing their raw data records. We can then expand top-k similarity search from a single dataset to the whole federation, via top-k vector search over the integrated vector database.

While this is desirable, it is however challenging to find such a function $\mathcal{T}$ for a few reasons.